

An AI meeting notetaker

Record a conversation and get back a synced transcript, an AI summary, action items, highlights, cross-meeting search, and a public clip link. Built as a 24-hour rebuild of fathom.video.

8	234	31	7.5k
SEEDED MEETINGS	TRANSCRIPT SEGMENTS	API ENDPOINTS	LINES OF CODE

The pipeline is real, not a mockup.

Meetly captures your microphone in the browser, stores the audio, transcribes it with Whisper, summarises it with an LLM, and cuts real clip files with ffmpeg. The eight seeded demo meetings are authored content and are badged as such everywhere they appear, so demo data is never mistaken for live AI output.

Stack	React 19 · TypeScript · Vite · Tailwind v4 FastAPI · SQLAlchemy · SQLite
AI	OpenAI Whisper (transcription) · GPT models (summaries) · deterministic local fallback
Media	MediaRecorder capture · HTTP Range streaming · ffmpeg clip cutting
Scope	Single-user workspace, no authentication — a deliberate decision, see section 11
Author	Muhammad Umar

Contents

1. What the product does	4
The end-to-end flow	4
Core surfaces	4
2. Architecture	4
The decision that kept the codebase small	5
Data model	5
Schema migrations without a migration tool	6
3. What is real and what is not	6
4. The AI layer	6
Two paths, one contract	7
Validating model output	7
The five templates	7
Configuration from the interface	7
5. Recording and the media pipeline	7
What happens when you press record	8
Processing states	8
Storage and serving	8
Clips are real files	8
6. Search	8
Measured coverage	9
7. Seed data	9
Highlights anchor to quotes, not timestamps	10
8. The eight-person, one-hour case	10
9. API reference	10
10. Engineering notes	11
A diagnosis worth recording	12
11. Product decisions	12
What was built first, and why	12
What was deliberately left out	12
Where it would go next	13

12. Verification	13
13. Security	13
14. Running it	14
Backend	14
Frontend	14
Enabling the AI features	14
Deployment	14
15. Known limitations	14

1. What the product does

Meetly is built around a single observation: the value of a meeting notetaker is not the recording, it is everything that happens after the call ends. The product is therefore organised around the post-meeting workspace rather than the capture mechanism.

The end-to-end flow

record → store → transcribe → analyse → play back
→ search → highlight → clip → share

Core surfaces

Surface	What it does
Dashboard	Recent meetings, upcoming calls and four metrics computed from the database. Shows a setup banner when no AI key is configured.
Meetings list	Filter by title, type or participant; sort by date; list or grid view; delete behind a confirmation.
Meeting detail	The centre of the product. Player, AI summary, action items and highlights, with the transcript locked in a rail beside them rather than below the fold.
Transcript	Speaker-grouped and timestamped. Click any line to seek; playback highlights the active line. In-meeting search with match count, copy and download.
AI summary	Overview, key points, decisions, topics and insights, with five switchable templates that genuinely re-read the transcript.
Action items	Task, assignee, due date and status. Add, edit, complete and delete, all persisted.
Highlights	Four categories, each carrying the transcript excerpt it came from, rendered as coloured markers on the timeline.
Search	One query across six surfaces, deep-linking to the exact second.
Clip sharing	Select a range, get a public link that opens with no account.
Calendar	Upcoming and past meetings with a simulated provider connection flow.
Settings	Profile, theme, notifications, meeting preferences and live AI configuration.

2. Architecture

```
backend/ FastAPI + SQLAlchemy + SQLite
app/api/ one router per resource (776 lines)
app/models.py 8 tables
app/schemas.py Pydantic request/response types
app/services/ (636 lines)
ai.py LLM summaries + local fallback
transcription.py Whisper + honest no-key notice
templates.py 5 summary templates
storage.py media storage abstraction
clipper.py ffmpeg cutting + duration probe
config.py runtime AI configuration
app/seed/ one module per meeting (1,028 lines)
seed.py rebuilds the database

frontend/ React + TypeScript + Vite + Tailwind
src/components/ UI primitives, layout, panels (2,183 lines)
src/pages/ one per route (1,570 lines)
src/hooks/usePlayer.ts dual-engine playback transport
src/hooks/useRecorder.ts MediaRecorder capture
src/lib/api.ts typed API client
```

The decision that kept the codebase small

Seeded meetings have no audio file; recorded meetings do. Rather than fork the interface into two versions, **usePlayer keeps one public interface and swaps its engine internally** — a real <audio> element when media exists, a virtual clock when it does not.

Every consumer — the transcript, highlights, summary panel, clip dialog and the public share page — is unaware of the difference. This is what made real playback a contained change instead of a rewrite, and it is why the eight seeded meetings kept working untouched.

Data model

Table	Purpose
Meeting	Title, date, duration, type, platform, media reference, processing state, provenance
Participant	Name, email, role, per meeting
TranscriptSegment	Speaker, start, end, text
Summary	Overview, key points, decisions, topics, insights, generation source
ActionItem	Task, assignee, due date, status, source timestamp
Highlight	Title, range, excerpt, speaker, category
Clip	Range, share token, optional cut media file
Setting	Key-value store for AI configuration set from the UI

Schema migrations without a migration tool

The project is small enough that Alembic would be more machinery than it earns. On startup, `ensure_columns()` compares the model metadata against the live database and issues `ALTER TABLE ADD COLUMN` for anything missing. Every added column is nullable or defaulted, so an existing database is upgraded in place with its data intact — verified against a database holding 13 meetings and 234 segments.

3. What is real and what is not

The product separates three kinds of content and labels them in the interface so they cannot be confused: **real** output from a live pipeline, **fallback** output when no API key is present, and **authored** demo content.

Capability	Status
Browser recording	Real. MediaRecorder captures the microphone, with a live input meter proving audio is arriving.
Media storage	Real. Written to disk under a generated filename, served with HTTP Range support so seeking works.
Playback	Real for recordings. Seeded meetings have no audio and use the virtual transport.
Transcription	Real with an API key (Whisper, real per-segment timestamps). Without one, nothing is transcribed and the transcript says so.
AI summary	Real. LLM with a key, local extraction without. Both validate the response before storing it.
Template switching	Real. Each template re-reads the transcript rather than relabelling sections.
Clip cutting	Real. ffmpeg cuts the range into its own file.
Public sharing	Real and genuinely public — no account required.
Search	Real. Database queries across six surfaces.
Seeded meetings	Authored. Written by hand so the product demonstrates well before anything is recorded. Badged "Demo data".
Recording bot	Not built. Meetly records the device, not a remote Zoom or Teams call.
Calendar OAuth	Simulated and clearly labelled, structured so a real provider drops in.
Authentication	None. A deliberate omission — see section 11.

Never present authored content as live output.

Every meeting carries a provenance badge: Demo data, Real recording, or Simulated capture. When no API key is configured, the transcript states plainly that nothing was transcribed rather than inventing a conversation, and the summary panel names which path produced the text you are reading.

4. The AI layer

`services/ai.py` exposes two functions: `summarise()` for a meeting's own summary and `apply_template()` for reading it back under a different lens. Both follow the same shape: try the model, fall back to local extraction, never fail.

Two paths, one contract

	With an API key	Without a key
Transcription	Whisper, real per-segment timestamps	Nothing. The transcript says so explicitly.
Summary	Model returns structured JSON, validated before storage	Longest contributions as key points, phrase matching for decisions, term frequency for topics
Templates	Model is asked for exactly that template's headings	Cue matching pulls the transcript lines each section is about
Citations	None — model output is paraphrase	Exact. Every point is a quoted line, so its timestamp is real

The fallback produces a genuinely useful artefact rather than a placeholder, and it has one property the model path does not: because each point is a verbatim transcript line, the timestamp attached to it is exact. Those points are clickable and play the moment they came from.

Validating model output

A model can return a string where a list belongs, or omit a key entirely. Storing that unchecked is how one bad response becomes a broken meeting page. Every response passes through a validator that requires an overview, coerces the four list fields to lists of non-empty strings, and drops malformed action items. A response that fails validation falls back rather than raising.

The five templates

Template	Sections it produces
General meeting	Key points, Decisions, AI insights — served from the meeting's stored summary
Sales call	Customer needs, Objections and risks, Commitments made, Next steps
Engineering review	Technical decisions, Trade-offs discussed, Risks and blockers, Follow-ups
One-on-one	Topics raised, Feedback, Owned next
Product review	User problems, Product decisions, Evidence cited, Open questions

Switching template changes the sections *and* their content. All 40 meeting-by-template combinations return successfully; the single combination that yields nothing — a sales discovery call read under the engineering template — shows an empty state saying so, which is the correct answer rather than an error.

Configuration from the interface

The API key and both models are set in **Settings** → **AI** and stored server-side in the `Setting` table. The key is never returned to the browser — only a masked form. The model dropdown is populated live from the account's own model list, filtered to chat-capable families with dated snapshots removed. A Test connection button verifies the key before anything depends on it.

5. Recording and the media pipeline

Meetly records **this device's microphone**, so everyone in the room is captured and remote participants are not. Building a bot that joins Zoom, Meet and Teams is days of infrastructure work and buys nothing the assignment is judged on.

What happens when you press record

```

1 getUserMedia() browser asks permission
2 MediaRecorder captures audio, live meter shows input level
3 POST /meetings/record multipart upload
4 storage.save() uuid filename, mime allowlist, size cap
5 Meeting created status = processing, returns immediately
6 BackgroundTasks transcribe → summarise → extract action items
7 status → ready, or failed with the reason attached
8 UI polls every 2s progress notice, then the finished meeting

```

Processing states

A meeting carries a `processing_status` of `processing`, `ready` or `failed`. Failures write the reason to the meeting and the detail page renders it. The interface never sits on an indefinite spinner: either it is working, it is done, or it says what broke.

Storage and serving

Concern	How it is handled
Filename	Generated UUID. The client's filename is never used — it is attacker-controlled and a path-traversal risk.
Type	Mime allowlist covering webm, ogg, mp4, mpeg and wav. Anything else is rejected with a 400.
Size	Capped by <code>MAX_UPLOAD_MB</code> , default 50 MB.
Seeking	Served through <code>FileResponse</code> , which honours HTTP Range. Without it the player would have to download the whole file before it could seek.
Duration	Probed from the media itself with <code>ffmpeg</code> rather than inferred from the transcript, which only covers speech.
Cleanup	Deleting a meeting removes its audio and any clip files. A startup sweep removes anything a file lock left behind.

Clips are real files

Creating a clip from a recording runs `ffmpeg` — supplied by the `imageio-ffmpeg` wheel, so nothing has to be installed on the host — and writes the selected range to its own file. The cut re-encodes rather than stream-copying: stream copy snaps to keyframes and can miss the requested start by seconds, which defeats the point of choosing a range. If `ffmpeg` is unavailable the clip falls back to a time range played against the parent recording, which is also what every seeded meeting does, since they have no audio of their own.

6. Search

One query runs across six surfaces: transcript text, meeting titles, summary overviews, action items, highlight titles and participant names. Results group by meeting, carry a badge naming which surface matched, and highlight the matched term in context.

Where a hit has a timestamp — transcript lines and highlights — clicking it opens the meeting and seeks to that exact second via a query parameter the detail page reads on mount.

Measured coverage

Query	Results	Across meetings
customer	10	6
API	4	3
deployment	4	2
pricing	4	2
deadline	2	2
roadmap	4	1

A corpus problem, not a search problem.

Three of those terms originally returned nothing. The meetings discussed deployments, deadlines and APIs at length without ever using those exact words. The fix was to correct the vocabulary in the transcripts rather than to loosen the matching, because a search that pretends to match is worse than one that honestly finds nothing.

7. Seed data

Seed data was treated as a feature, not filler. Eight meetings with 234 transcript segments, written as real conversations with a continuing cast and a storyline that runs across them: a summarisation bug found in one meeting drives a roadmap decision in another, gets estimated in sprint planning, and is disclosed to investors. Cross-meeting search is only interesting if the corpus has something in it.

Meeting	Type	Platform	Len	People	Actions	Highlights
AI Platform Architecture Review	Internal	Zoom	58m	8	6	5
Product Strategy Sync	Internal	Meet	38m	6	5	4
Sprint Planning — Sprint 34	Internal	Meet	38m	7	4	3
Design Review — Meeting Workspace	Internal	Meet	32m	6	4	3
Customer Feedback — Helio Health	External	Teams	30m	5	4	4
Client Discovery — Northwind	External	Teams	30m	5	4	4
Investor Update — Q3	External	Zoom	24m	5	3	4
Weekly Engineering Standup	Internal	Zoom	16m	6	4	3

Highlights anchor to quotes, not timestamps

Transcript times are generated from speaking pace, so a hand-written timestamp drifts the moment any wording changes. This was not hypothetical: the first version placed highlights by timestamp and several landed on the wrong line. Seeded highlights are now anchored to a **quote** from the line they refer to, and the seeder resolves it to a real time. A stale anchor raises at seed time rather than shipping a highlight that plays the wrong moment.

Five upcoming meetings are also seeded, so the calendar and dashboard have something forward-looking to show.

8. The eight-person, one-hour case

The brief singles out a long multi-participant call as the case that actually matters. **AI Platform Architecture Review** is built for it.

8 PARTICIPANTS	58 MINUTES	56 SEGMENTS	11 ITEMS EXTRACTED
--------------------------	----------------------	-----------------------	------------------------------

At that length the interface problems are real rather than theoretical: the transcript cannot be an undifferentiated wall, speakers must be visually grouped, the summary needs structure, and action items need owner attribution across eight people. Consecutive lines from one speaker are therefore grouped under a single name, the way a real transcript reads.

The content carries its weight too. The meeting resolves three genuine engineering decisions — chunking strategy, vector store build-versus-buy, and a per-meeting cost ceiling — across six topic blocks, producing six action items and five highlights drawn from the discussion.

9. API reference

Method	Path	Purpose
GET	/api/health	Liveness
GET	/api/meetings	List meetings, filtered by status
POST	/api/meetings	Create a meeting
GET	/api/meetings/{id}	Full meeting: transcript, summary, items, highlights
DELETE	/api/meetings/{id}	Delete a meeting and its media
POST	/api/meetings/record	Upload a browser recording, start processing
GET	/api/meetings/{id}/media	Stream the recording, Range supported
GET	/api/meetings/{id}/transcript	Transcript segments
GET	/api/meetings/{id}/summary	Summary under a template
GET	/api/meetings/{id}/action-items	List action items
POST	/api/meetings/{id}/action-items	Create an action item

Method	Path	Purpose
PATCH	/api/action-items/{id}	Update task, assignee, due date or status
DELETE	/api/action-items/{id}	Delete an action item
GET	/api/meetings/{id}/highlights	Highlights for one meeting
POST	/api/meetings/{id}/highlights	Create a highlight
GET	/api/highlights	Highlights across all meetings
DELETE	/api/highlights/{id}	Delete a highlight
GET	/api/search	Search six surfaces
POST	/api/clips	Cut a clip and mint a share token
GET	/api/clips/{token}	Public clip payload
GET	/api/clips/{token}/media	Public clip audio
GET	/api/calendar	Upcoming and past meetings
POST	/api/calendar/connect	Simulated provider connection
POST	/api/recordings/start	Begin a simulated recording
POST	/api/recordings/{id}/stop	Finish a simulated recording
GET	/api/templates	Available summary templates
GET	/api/settings/ai	Current AI configuration, key masked
PUT	/api/settings/ai	Save key and model choices
GET	/api/settings/ai/models	Chat models available to the account
POST	/api/settings/ai/test	Verify the configured key
GET	/api/stats	Dashboard metrics

Interactive documentation is served at /docs.

10. Engineering notes

A record of problems found during development and how they were resolved. Several were caught only because the feature was tested rather than assumed to work.

Problem	Resolution
Seeded highlights played the wrong transcript line, because hand-written timestamps drifted against generated segment times.	Anchor highlights to a quote and resolve it at seed time. A stale anchor now fails loudly instead of shipping silently.
Deleting a meeting left its audio on disk. On Windows the unlink also raised WinError 32 while a player held the file, aborting the request and leaving the meeting undeleted.	Delete the rows first, then remove files best effort, plus a startup sweep for anything a lock left behind.

Problem	Resolution
Grouping transcript lines by speaker removed the per-line seek affordance.	The whole line is clickable, guarded by a selection check so text can still be selected.
On a short mobile stage the live caption overlapped the speaker name.	The stage became a flex column, so the two can no longer collide.
The sticky page header slid underneath the fixed mobile top bar.	Offset the sticky position below the large breakpoint.
A 25-second recording displayed as "0 min".	Show seconds below a minute.
The model filter dropped gpt-4.1, because it excluded anything ending in a digit to remove dated snapshots.	Match the date pattern explicitly instead.
Three search terms named in the brief returned nothing.	Corrected the vocabulary in the transcripts rather than loosening the matching.

A diagnosis worth recording

After adding the key to a file, transcription still reported no key. The cause was not the code: the server process had started roughly a minute before the file was written, so it had already loaded its environment. Reload watches Python files, not environment files. This is part of why configuration later moved into the interface, where the application can always report its own state truthfully.

11. Product decisions

What was built first, and why

The meeting detail page, before anything else. It is where the product is used and where a rebuild either feels real or does not. The dashboard, calendar and settings pages came last, because they are navigation around the thing rather than the thing itself.

What was deliberately left out

- **Authentication.** The brief requires the live link to open for someone not signed in. A login wall works directly against that and earns nothing against the stated criteria. No ownership column exists in the schema; adding one is the migration path.
- **A recording bot.** Reliably joining Zoom, Meet and Teams is days of infrastructure work, not product work. Recording the device exercises the entire pipeline honestly.
- **Citations on the authored summary.** Template-generated points *do* cite exact timestamps, because each is a quoted transcript line. The hand-written summary does not, because its points are paraphrases — and fuzzy-matching a paraphrase back to a line produces confident jumps to the wrong moment. A wrong citation is worse than none.
- **Semantic search.** Keyword search across six surfaces ships and is useful. Embeddings plus a vector index is a week, not an afternoon.
- **A background job queue.** FastAPI background tasks are sufficient in-process. Celery, Redis and a broker would be infrastructure the product does not yet need.

Where it would go next

In order: summary citations with segment ids carried through the pipeline; chunked summarisation so long meetings do not degrade; speaker diarization; object storage for media; then shared highlights, which the seeded customer interview argues is the highest-leverage missing feature.

12. Verification

Results below are measured, not asserted. Each was run against the working application.

Check	Result
Range requests	Pass. Returns 206 Partial Content with a correct Content-Range header — this is what makes seeking work.
Real playback	Pass. Audio played to 2.42s, seeked to 18.55s, and the interface clock followed to 0:18, proving the media element drives the UI.
Virtual transport	Pass. A seeded meeting has no audio element and its clock still advances, so demo meetings are unaffected by the media work.
Whisper transcription	Pass. Real speech returned four segments with accurate timestamps.
LLM summary	Pass. Produced an overview plus action items with owners and due dates from a real transcript.
Templates	Pass. 40 of 40 meeting-by-template combinations return successfully; the one empty result is correct and shows an empty state.
Clip cutting	Pass. A 10-second clip produced a 99 KB file from a 309 KB source.
Public share	Pass. The share link opens in a fresh session with no account.
Upload validation	Pass. A text file posing as audio is rejected with a clear 400.
Permission denied	Pass. Blocking the microphone produces an explicit message, not a silent empty recording.
Schema migration	Pass. Six columns added to a live database with 13 meetings and 234 segments intact.
Delete cascade	Pass. Removes the meeting, its media and its clip files; confirmation required.
Secret scan	Pass. No API key in any tracked file, in the full commit history, or in the agent logs.
Build	Pass. TypeScript compiles clean; production bundle 118 KB gzipped.
Console	Pass. No errors across every route in a clean session.

13. Security

Area	Position
Secrets	No key is committed. The .env file and media directory are gitignored; the full commit history was scanned.
Key handling	Stored server-side and never returned to the browser — only a masked form for confirmation.
Uploads	Generated filenames, a mime allowlist, a size cap, and a path check that prevents escaping the media directory.

Area	Position
Share tokens	Generated with the secrets module, long enough not to be guessable, and scoped to a single clip.
Public surface	The share route exposes only the clip, its transcript lines and participant names — not the parent meeting.
CORS	Restricted to a configured origin list rather than a wildcard.
Known gap	No authentication. Anyone who can reach the application can use a saved key. On a public deployment, supply the key through the server environment instead. This is stated in the interface, not hidden.

14. Running it

Two terminals. Python 3.11+ and Node 18+.

Backend

```
cd backend
pip install -r requirements.txt
python seed.py
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173`. Vite proxies the API, so no environment variables are needed for local development.

Enabling the AI features

The OpenAI key is configured from **Settings** → **AI** inside the running app, not from a file. Paste the key, choose a summary model from the list fetched live from your account, and press Test connection to confirm it works before relying on it.

Deployment

Backend and frontend deploy independently: a `render.yaml` for the API and a `vercel.json` for the client, including the rewrite that keeps a share link working on a hard refresh. The API seeds itself on first boot if its database is empty, so a fresh deployment is never an empty workspace. Recording requires a secure context, so the deployed site must be served over HTTPS or the browser will refuse the microphone.

15. Known limitations

- **No speaker diarization.** The Whisper API does not return speakers, so every segment of a recording carries a single label. Seeded meetings have true multi-speaker transcripts because they were authored.

- **Recordings are local files.** On a host with an ephemeral disk they do not survive a redeploy. The storage layer is three functions; pointing them at object storage is the fix.
- **Recording requires a secure context.** Browsers only grant microphone access on localhost or HTTPS.
- **Re-seeding is destructive.** seed.py drops every table, which removes recordings along with the demo data.
- **Whisper and the LLM are billed per use.** Only meetings you record yourself reach them.
- **Long meetings are summarised in one pass.** Very long transcripts would degrade; chunked summarisation is the planned fix, and is itself the subject of one of the seeded meetings.